



BLUE BITS

البرمجة التفرعية

نظري (4)

ملف داعم

2026/2025

السنة الرابعة



مكتبة شهاب
كشياء الذائفة - يفرق يفرق سالي الفاني

[f](#) [@](#) [v](#) [g](#) /BlueBitsTeam

www.bluebitsteam.com



تذكرة أنماط التواصل بين ال *processes*:

1. *point-to-point*

2. *collective*

نمط ال *collective*:

- يتميز هذا النمط بغياب ال *tags* من الرسائل
- العمليات الأساسية فيه (*principal operations*):

↳ *MPI_Bcast()*

تمثل عملية *broadcast* من ال *process* المرسلة الى كل ال *processes* الأخرى.

↳ *MPI_Reduce()*

تمثل عملية دمج القيم لكل ال *processes* في قيمة واحدة.

↳ *MPI_Gather()*

تمثل عملية تجميع القيم لمجموعة جزئية من ال *processes* في قيمة واحدة.

↳ *MPI_Scatter()*

تمثل عملية تقسيم ال *buffer* الى عدة أجزاء وذلك من أجل مجموعة جزئية من ال *processes*.

↳ *MPI_Alltoall()*

تمثل عملية ارسال بيانات من كل ال *processes* الى كل ال *processes*.

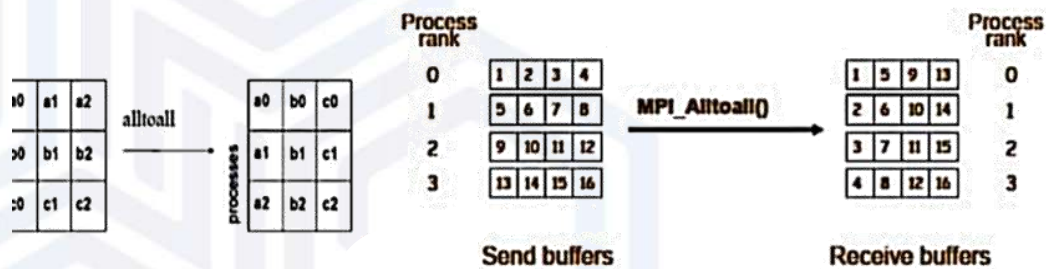
↳ *MPI_Reduce_scatter()*

تمثل عملية دمج القيم وتقسيم النتائج.

MPI_ALLTOALL:

كل عملية ترسل البيانات إلى كل العمليات الأخرى.

MPI_ALLTOALL(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, comm)

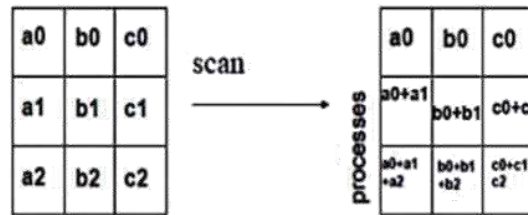


**MPI_SCAN :**

- كل عملية تقوم بتخزين تراكمي وتطبيق العملية المطلوبة على نفسها وكل شيء قبلها (أصغر منها رتبة)
- كل *process* ذات رتبة $rank = i$ تحتوي نتائج الـ *reduction* للبيانات من الـ *processes* ذات الرتبة $rank = 0$ وحتى *process* ذات الرتبة $rank = i$

*Int MPI_Scan(void * sendbuf, void * recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)*

Process (i) will contain the results of reduction of data from processes 0..i

**Status parameter :**

- يمكن للمهمة المستقبلية أن تقوم بعملها دون معرفة قيم المعلومات التالية:

1. المهمة المرسل للرسالة.
2. علامة الرسالة *tag*.
3. وحتى حجم البيانات في الرسالة.

- فإذا كيف للمستقبل أن يعرف هذه المعلومات؟

نذكر البارامتر الأخير في تعليمة الاستقبال كان *MPI_Status* ، وهو عبارة عن سجل *struct* يحتوي على ثلاثة حقول هي:

1. *MPI_SOURCE* : يحتوي على رقم المهمة المرسل.
2. *MPI_TAG* : يحتوي على علامة الرسالة.
3. *MPI_ERROR* : رقم الخطأ في حال حصول خطأ ما.

- لنقم الآن بكتابة برنامج أكثر تعقيداً وفائدة من البرامج السابقة، سنقوم الآن بجمع قيم مصفوفة أحادية البعد بشكل تفرعي. لنفرض أن المصفوفة تتألف من عشرة عناصر المطلوب توزيع العمل على جميع المهمات بحيث تقوم كل مهمة بجمع جزء من المصفوفة ثم تقوم بإرسال النتيجة للمهمة رقم صفر.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----



لو أردنا تحليل البرنامج فيمكن تقسيم البرنامج إلى ثلاث مراحل:

- المرحلة الأولى هي مرحلة توزيع البيانات.
- المرحلة الثانية هي مرحلة حساب النتائج.
- المرحلة الثالثة هي مرحلة جمع النتائج من المهمات.

```
1  #include <stdio.h>
2  #include <mpi.h>
3  #define N 10
4  void main()
5  {
6  int rank, size, data[N];
7  MPI_Status status;
8  MPI_Init(NULL, NULL);
9  MPI_Comm_size(MPI_COMM_WORLD, &size);
10 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11     if(rank == 0)
12     {
13         //array initialization
14         for(i=0;i<N;i++)
15         {
16             data[i] = i + 1;
17         }
18         //print the data array
19         print("data array is:\n");
20         for(i = 0; i < N; i++)
21         {
22             print("%d\t", a[i]);
23         }
24         print("\n");
25
26         /** data distribution */
27         for(i=1; i<size; i++)
28             // ممنوع تبدأ الحلقة من 0 لأنها ممنوع ترسل لنفسها لكي لا تبقى ب dead lock
29         {
30             // المصفوفة المراد إرسالها، N : عدد العناصر
31             MPI_Send(&data, N, MPI_INT, i, 0, MPI_COMM_WORLD);
32         }
33     } //end if statement {process 0 section}
34     else //for other processes
35     {
36         MPI_Recv(&data, N, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
37         // tag ، سوف استقبل من 0
38
39         /** Computation */
40         int sum = 0;
41         for(i = rank * N/size; i < (rank + 1) * N/size; i++)
```



```

42 {
43  /** إلى الإرسال يتم الجمع وبعد العناصر جمع0/**
44    sum = sum + data[i];
45  }
46  /** result gathering **/
47  if(rank != 0)
48  /** 0 فقط التخزين لأن ترسل لازم 0 ليست إذا /**
49  {
50    MPI_Send(&sum, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
51  }
52  else //for process0
53
54  // التخزين ثم النتائج جمع ثم بالاستقبال تقوم process 0 حيث
55  {
56    int total, partial_sum;
57    total = sum;
58    for(i = 1; i < size; i++)
59    // وهكذا والثانية الأولى من معالجات 5 من تستقبل سوف
60    {
61      MPI_Recv(&partial_sum, 1, MPI_INT, i, 0, MPI_COMM_WORLD, &status);
62      total = total + partial_sum;
63    }
64    printf("sum of matrix is: %d", total);
65  }
66  MPI_Finalize();
67 }

```

ملاحظة:

$$first = \frac{rank * n}{size} + 1$$

لأن المصفوفة دائماً تبدأ من 0

$$i < last = \frac{(rank + 1) * N}{size}$$

مثلاً لو لدي 100 عنصر لنصل للـ 99 نضع أصغر تماماً

- يمثل i هنا $index$ عنصر المصفوفة الذي ستأخذه العملية التي رتبها $rank$

مثال: $rank = 2$

$$i = 2 \times 10 / 5 = 4$$

أي سوف تبدأ $process2$ من الـ $index = 4$

قبل أن نقوم بمناقشة البرنامج لنرى نتيجة تنفيذه:

```
Data array is:
1  2  3  4  5  6  7  8  9 10
sum of matrix is: 55
```

ملاحظة:

كل معالجة سوف تأخذ عدد معين من العناصر وتجمعهم مثلاً: $p = 5, N = 10$
فكل معالجة ستأخذ عنصرين وتجمعهم.

- عندما أرسل للكل في *point to point* ضروري *for* فيها *send* عدد العناصر المرسل (i)

توزيع البيانات:

قمنا في بداية البرنامج (السطر 17 - 14) بتهيئة مصفوفة البيانات، حيث تمت تهيئة المصفوفة من قبل المهمة رقم صفر، وبعدها المهمة رقم صفر ستقوم بتوزيع البيانات (المرحلة الأولى توزيع البيانات) حيث قامت المهمة رقم صفر بإرسال مصفوفة البيانات إلى بقية المهمات.

معالجة البيانات:

بعد أن تم توزيع البيانات يجب أن تقوم كل مهمة بحساب مجموع الجزء الخاص بها من المصفوفة (السطر 46 - 41)، طالما أن كل المهمات لديها مصفوفة البيانات كاملة فيجب أن تعرف كل مهمة المجال الذي يجب أن تعمل عليه، ولكن كيف ستعرف كل مهمة مجال عملها؟! حسناً أولاً لنفكر ما هو حجم عمل كل مهمة!!

جمع النتائج:

بعد أن تمت معالجة البيانات وأصبحت كل مهمة لديها نتيجة الجمع الجزئية ضمن متحول *sum* فإنه يجب على كل مهمة أن تقوم بإرسال النتائج للمهمة رقم صفر، بحيث تقوم المهمة رقم صفر باستقبال هذه النتائج الجزئية وجمعها بعضها مع بعض مع نتيجة جمعها الجزئية نفسها بحيث تنتج في النهاية نتيجة جمع جميع عناصر المصفوفة.

لو فكرنا قليلاً لوجدنا أن كل مهمة سيكون حجم العمل الخاص بها ينتج من تقسيم حجم العمل الكلي على عدد المهمات الكلي، طبعاً هنا حجم العمل الكلي هو حجم المصفوفة، وبالتالي فإن حجم العمل يساوي $N/size$. فإذا كان حجم المصفوفة الكلي 10 وكان عدد المهمات 5 فينتج معنا أن كل مهمة ستقوم بمعالجة حجتين فقط من المصفوفة.

إذا عرفنا الآن ما هو حجم العمل لكل مهمة، بقي علينا الآن معرفة ما هو المجال من المصفوفة الذي ستعمل عليه كل مهمة، فلو كان حجم المصفوفة 10 وعدد المهمات 5 فإن المهمة رقم صفر ستعمل على أول حجتين والمهمة رقم 1 ستعمل على الحجرة الثالثة والرابعة وهكذا، ولكن كيف سنحدد أن المهمة رقم صفر سوف تعالج أول عنصرين من المصفوفة، والمهمة رقم واحد ستعالج العنصرين الثالث والرابع؟! لو فكرنا قليلاً يمكن من خلال رقم المهمة rank أن نحدد ماهي العناصر التي ستعمل عليها المهمة، فمن خلال ضرب رقم المهمة بحجم العمل ينتج معنا رقم أول عنصر من المصفوفة التي ستعمل عليه المهمة، وستنتهي قبل بداية مجال المهمة التالية، كما هو موضح في السطر 40:



```
for( i = rank * N/size; i < (rank + 1) * N/size; i ++)
```

بداية مجال
المصفوفة

نهاية مجال
المهمة الحالية
والتي هو نفسه
بداية مجال المهمة

لنمر الآن مثلاً ليوضح معنا بشكل أفضل، كما قلنا حجم المصفوفة $N = 10$ ولو فرضنا أن عدد المهام $size = 5$ ، فإذاً سيكون مجال المهمة رقم صفر هو:

$$start_index = rank * N/size = 0 * 10/5 = 0;$$

$$end_index = (rank + 1) * N/size = (0 + 1) * 10/5 = 2;$$

فإذاً سيبدأ عمل المهمة رقم صفر من الخانة رقم صفر وينتهي قبل الخانة رقم 2 أي $[0 - 1]$ ، وهكذا بالنسبة لبقية المهام.

مثال: *Message tags*

```
if (rank == 0) {
    int N=12;
    int M=29;
    // Message A
    MPI_Send(&N, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    // Message B
    MPI_Send(&M, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);
}
else if (rank == 1) {
    int A=0;
    int B=0;
    MPI_Status Stat; //Struct of important information
    // Want to receive message B
    MPI_Recv(&B, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, &Stat);
    printf("B: Got %d from master with tag %d\n", B, TAG2);
    // Want to receive message A
    MPI_Recv(&A, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &Stat);
    printf("A: Got %d from master with tag %d\n", A, TAG1);
}
```



إرسال A ثم B ، وبعد ذلك استقبال B ثم استقبال A .

مثال 2 : Data Movement

هنا المطلوب جمع N عدد ولكن ليست مصفوفة.

```
1  if (myid == 0) {
2      // في وتضعه الثاني البارامتر تأخذ سوف 0 رقم process ال
3      N = atoi(argv[1]); //get the number that the user wants
4      for (i = 1; i < numproc; i++) { //Send to all nodes
5          MPI_Send(&N, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
6      }
7      sum0 = 0;
8      // p0 للعملية لجزئي ا الجمع بعملية تقوم
9      for(i = 1; i <= N/numproc; i++) sum0 = sum0 + i;
10     // 0 ال في توضع الجمع بعد
11     result = sum0;
12     for (i = 1; i < numproc; i++) { //receive from all nodes
13         MPI_Recv(&sum1, 1, MPI_INT, i, 0, MPI_COMM_WORLD, &Stat);
14         result = result + sum1; }
15     // طباعة ثم جمع ثم الكل من استقبلت
16     fprintf (stdout, "The sum from 1 to %d is %d \n", N, result);
17 }
18 else { // تكون لن معالجة 0
19     MPI_Recv(&N, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &Stat);
20     sum1 = 0;
21     for(i = (N/numproc * myid) + 1; i <= (N/numproc * (myid + 1)); i++)
22         sum1 = sum1 + i;
23     MPI_Send(&sum1, 1, MPI_INT, 0, 0, MPI_COMM_WORLD); } //
This can be simplified by collective broadcasting.
```

ملاحظة:

$process 0$ تقوم بإرسال القيم لل $processes$ الباقية،
وباقى ال $processes$ تقوم بالاستقبال ومن ثم تجمع العناصر
وترسلها إلى $process 0$

مثال 3: *MPI_Alltoall* & *MPI_BARRIER*

- الكل يرسل للكل أي الناتج مثل منقول مصفوفة.

```
1. int *recvdata; int *senddata;
2. int numproc, myid, i, N, ndata;
3. MPI_Init(&argc, &argv);
4. MPI_Comm_size(MPI_COMM_WORLD, &numproc);
5. MPI_Comm_rank(MPI_COMM_WORLD, &myid);
6. senddata = (int *)malloc(numproc * sizeof(int));
7. recvdata = (int *)malloc(numproc * sizeof(int));
8. // المعالجات مصفوفة التهيئة
9. for (i = 0; i < numproc; ++i)
10. senddata[i] = 1 + myid * numproc + i;
11. printf("Before ID = %d : ", myid);
12. for (i = 0; i < numproc; ++i)
13. printf(" %d", senddata[i]);
14. ndata = 1; // والاستقبال بالإرسال نفسها) البيانات عدد
15.
16.
17. MPI_Alltoall (senddata, ndata, MPI_INT, recvdata, ndata, MPI_INT, MPI_COMM_WORLD);
18. MPI_Barrier(MPI_COMM_WORLD);
19. // فارغة قيم يطبع عملها من المعالجة تنتهي لم إذا لأن Barrier استخدمت
20. printf("After ID = %d : ", myid);
21. for (i = 0; i < numproc; ++i)
22. printf(" %d", recvdata[i]);
23. MPI_Finalize()
```

ملاحظات:

بارامترات الـ *malloc* مصفوفة حجمها عدد *processes* مضروبة بعدد البايتات التي تحتاجها
تحتاج 4 بايتات وبعثت 5 عناصر ($4 * 5 = 20$ حجرة) *int* مثلاً:
علاقة تهيئة المصفوفة إما يتم إعطاؤها بالنص $senddata[i] = 1 + myid * numproc + i$
أو يتم حفظها بالشكل التالي: $senddata[i] = myid$

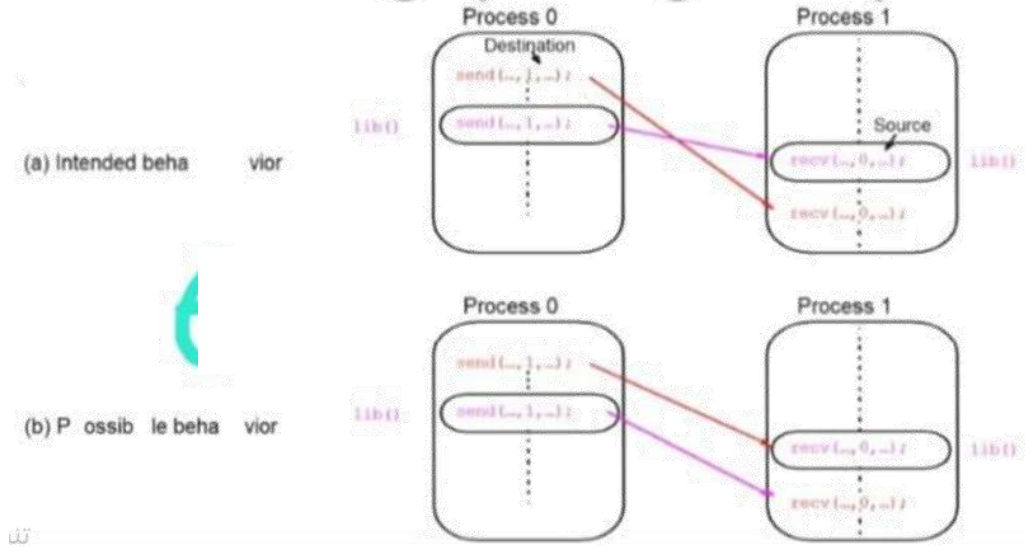
MPI: Point to Point Communication

- يستخدم إجراءات الإرسال والاستقبال مع *message tags*
- يمكن أن يستخدم المستقبل هذه الـ *wild card*:
لـ *MPI_ANY_TAG* إن لم تكن نعرف الـ *tag*

- **MPI_ANY_SOURCE** إذا لم نكن نعرف الـ **source** أو لا نريد تحديد هدف محدد نوع الـ **data** التي يمكن أخذها من مكتبة **MPI**

(**MPI_INT, MPI_FLOAT, MPI_CHAR**)

يتم اشتقاق أنواع البيانات المحددة من قبل المستخدم من أنواع البيانات الموجودة

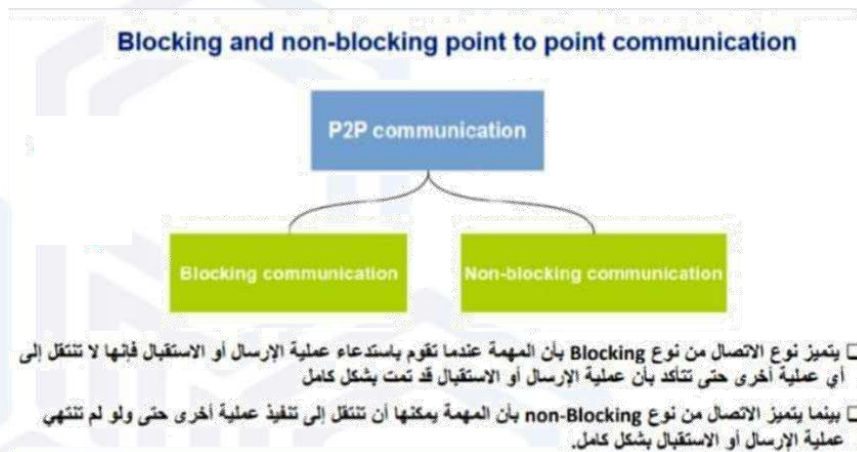


MPI: COMPLETION

يوجد العديد من الإصدارات للـ **receive & send**:

Locally Complete & Globally Complete

- يكون الروتين **Locally Complete** إذا كانت قد أكملت جميع أجزاءها في العملية
- يكون الروتين **Globally Complete** إذا اكتملت جميع أجزاء العملية لجميع الأطراف المشاركة في العملية وتم إجراء العملية بأكملها



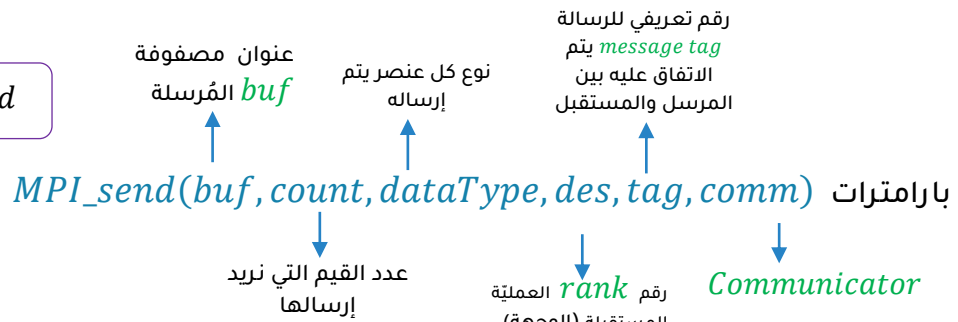
MPI: Blocking Routing

تعيد روتينات الإرسال والاستقبال الحجزية (**Blocking**) عند الانتهاء المحلي منها **Locally completed**.

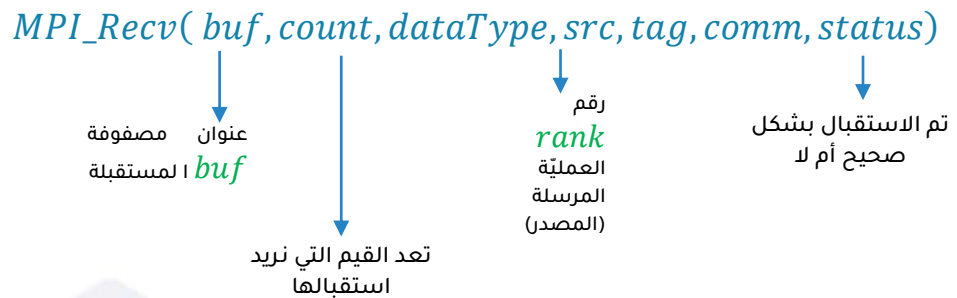


- عندما يمكن استخدام المكان الذي يحمل الرسالة مرة أخرى أو تغييره دون التأثير على الرسالة المرسل.
- ❖ تقوم روتينات الإرسال الحجزية (*Blocking send*) بإرسال الرسالة والعودة بدون الضمان بأن الرسالة قد تم استلامها، فقط يعني أن العملية حرة للمضي قدماً دون التأثير على الرسالة بشكل سلبي.
- ❖ تقوم روتينات الاستقبال الحجزية (*Blocking Receive*) بإرجاع العملية عند الانتهاء المحلي منها.
- تم استلام الرسالة في المكان المقصود ويمكن قراءة المحتوى الموجود في المكان المقصود بعد الاستقبال.

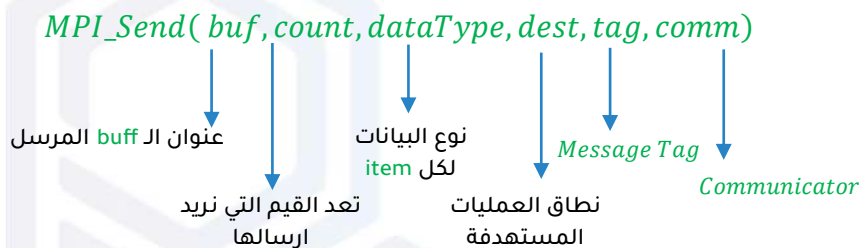
Parameters of blocking send



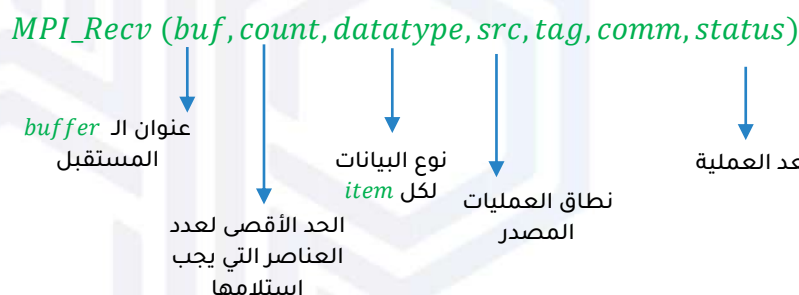
Parameters of blocking receive



Parameters of Blocking Send



Parameters of Blocking Receive





مثال: لإرسال عدد $int\ x$ من p_0 إلى p_1

```
1. int x;
2. MPI_Comm_rank(MPI_COMM_WORLD, &myrank); /* find rank */
3. if (myrank == 0) {
4.   MPI_Send(&x, 1, MPI_INT, 1, msgtag, MPI_COMM_WORLD);
5. }
6. else if (myrank == 1) {
7.   MPI_Recv(&x, 1, MPI_INT, 0, msgtag, MPI_COMM_WORLD, status);
8. }
```

MPI: Nonblocking Routines

تقوم الروتينات غير الحجزية (*Non – Blocking*) بإرجاع العملية فوراً.

- وبذلك يتمكن البرنامج من تنفيذ الأمر التالي بغض النظر عما إذا كان الروتين قد اكتمل محلياً أم لا.
- تقوم الروتينات غير الحجزية بإرجاع العملية فوراً، مثل:
- الإرسال غير الحجزى - *MPI_Isend()* - حيث يتم إرجاع العملية فوراً حتى قبل أن يتم تأمين مكان المصدر.
- الاستقبال غير الحجزى - *MPI_Irecv()* - حيث يتم إرجاع العملية فوراً حتى لو لم يتم استقبال أي رسالة.

ملاحظة:

حرف **a** في التوابع السابقة تدل على أنَّ العملية غير متزامنة (من نوع *blocking*)

- صيغ *Nonblocking Routine* هي:

MPI_Isend(buf, count, datatype, dest, tag, comm, request)

MPI_Irecv(buf, count, datatype, source, tag, comm, request)

request: يتم وضع أن الطرف الثاني قام بالاستقبال أم لا.

- يمكن اكتشاف الانتهاء باستخدام روتينات منفصل *MPI_Test* و *MPI_Wait*

تقوم روتين *MPI_Wait()* بالانتظار حتى يتم الانتهاء من العملية ثم تعيد البرنامج.

تقوم روتين *MPI_Test()* بإرجاع العلامة (*flag*) على الفور، مشيرة إلى ما إذا كانت العملية قد اكتملت في ذلك الوقت أم لا.

- تحتاج هذه الروتينات إلى ربطها بعملية معينة ويتم ذلك باستخدام نفس المعامل (*request*) في كليهما.



مثال: لإرسال عدد $int\ x$ من p_0 إلى p_1 والسماح لـ p_0 بالاستمرار

```
1. int x;
2. MPI_Comm_rank(MPI_COMM_WORLD, &myrank); /* find rank */
3. if (myrank == 0) {
4.   MPI_Isend(&x, 1, MPI_INT, 1, msgtag, MPI_COMM_WORLD, req1);
5.   Compute();
6.   MPI_Wait(req1, status);
7. }
8. else if (myrank == 1) {
9.   MPI_Recv(&x, 1, MPI_INT, 0, msgtag, MPI_COMM_WORLD, status);
10. }
```

تعليمات الإرسال والاستقبال من نوع Non – Blocking

عملية الإرسال *MPI_Isend*

```
1. MPI_Isend(void * buffer, // إرسالها المراد البيانات عنوان
   int count, // إرسالها المراد العناصر عدد
   MPI_Datatype datatype, // نوع البيانات
   int dest, // رقم المهمة الهدف
   int tag, // رقم الرسالة
   MPI_Comm comm, // اسم الحاوية
   MPI_Request * request) // حالة الإرسال
```

جميع البارامترات تقريباً رأيناها سابقاً في تعليمة الإرسال *MPI_Send*، يوجد فقط بارامتر واحد جديد هو *request* مهمة حيث يعتبر بارامتر خرج، حيث يتم فيه تخزين حالة الإرسال (هل انتهى الإرسال أم لا؟)، بحيث يمكننا فيما بعد اختبار اكتمال عملية الإرسال من خلال هذا البارامتر.

عملية الاستقبال *MPI_Irecv* تعليمة الاستقبال لها الشكل التالي:

```
1. MPI_Irecv(void * buffer,
2. int count,
3. MPI_Datatype datatype,
4. int src, // رقم المهمة المصدر
5. int tag,
6. MPI_Comm comm,
7. MPI_Request * request)
```

نلاحظ أن بارامترات هذه التعليمة مطابقة تماماً لتعليمة الإرسال، مع ملاحظة عدم وجود بارامتر *status* الذي رأيناها في تعليمة *MPI_Recv* حيث يمكننا الاستعلام والحصول على قيمة *status* بعد أن ينتهي الاستقبال بالكامل من خلال تعليمة *MPI_Wait*

2026/2025

السنة الرابعة



أعضاء الفريق

الفريق التقني

إيناس بيلوني	ظلال بابا
تسنيم طالب	عائشة قلّه
حلا إبراهيم	عائشة مُصطفى
دلال تسقيه	قمر حَمَد
ديانا صَبَّاغ	محمد يمان جلال
زهير حَقّامي	محمود غزال
شكرية سُلطان	وفاء حَقّامي
شهد كرّوم	يوسف نيّال

الفريق العلمي

إبراهيم هبرة	عائشة العُمر
إسراء حاج حمدي	العلّوش
بتول لبّان	عمر قَطّان
بشرى كوكش	فاطمة الحسين
بنيامين حكيمان	قاسم يحيى
تسنيم حوكان	كريست ياغجيان
تسنيم نور الدّين	محمد منير
جميل هلال	مديحة جَمّال
جودي جلال	ميرا فضلي
خديجة بيازيد	نجم الدّين درويش
رشا معراوي	هبة الله كتلو
سدرة مبيض	وداد قلعيّة
	ياسر صَبّاغ

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

